

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 8

The Same Calls, Inside Your Own OS

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: Process Creation in xv6 (run.c) 2
2.1.1 Observe Answers (Part 1) 2
2.2 Part 2: Inter-Process Communication (pingpong.c) 3
2.2.1 Observe Answers (Part 2) 3

3 CONCLUSIONS **4**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The main purpose of this laboratory assignment is to analyze the behavior of process control system calls (specifically `fork`, `exec`, and `wait`) running natively within the xv6 educational operating system. Furthermore, it aims to contrast these calls against standard Linux counterparts and to establish a two-way communication channel between processes using a dual-pipe mechanism in the `pingpong.c` program.

1.2 Theoretical Background

The kernel of xv6 offers a simplified implementation of POSIX-compliant system calls:

- **Managing Processes (`fork`, `exec`, `wait`):** A new child process is spawned by `fork()` through the duplication of the parent's kernel state. The `exec()` system call subsequently reads an ELF binary from the disk, overwriting the child's memory space. Finally, `wait()` is used by the parent to halt its execution until the child process completes its task.
- **One-Way Pipes (`pipe`):** Pipes function as memory buffers managed by the kernel. Since data flows in only one direction per pipe, establishing a bidirectional dialogue between two processes requires the creation of two distinct pipes (e.g., one for parent-to-child `p2c`, and another for child-to-parent `c2p`).
- **Blocking I/O Reads:** If a process attempts a `read()` on a pipe that contains no data, the OS puts the caller to sleep until another process writes data into the pipe using `write()`.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: Process Creation in xv6 (run.c)

The custom `user/run.c` program illustrates the typical three-step command execution cycle inside the xv6 environment.

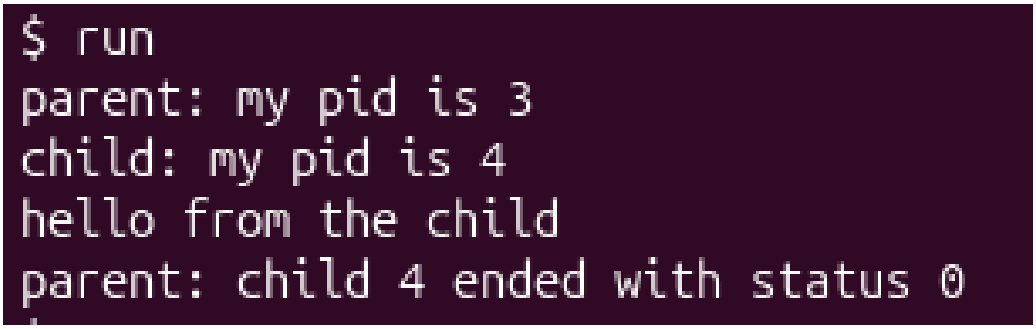


Figure 2.1: Execution of run.c in xv6 with valid and non-existent programs

2.1.1 Observe Answers (Part 1)

1. **Comparing System Calls:** In xv6, the commands used are `fork()`, `exec()`, and `wait()`, whereas a Linux environment relies on `fork()`, `execlp()`, and `waitpid()`. Despite the naming differences, the underlying principles—creating a clone, replacing the execution image, and awaiting completion—are entirely analogous.

2. **Source of the Output:** The phrase `"hello from the child"` originates from the `echo` program, which prints it only after the `exec()` call has successfully replaced the child process's memory.

3. **Differences in Exit Codes:** The exit code returned to the waiting parent is directly determined by the value passed to `exit()`. In the xv6 system, if `exec` fails, `user/run.c` is coded to terminate with `exit(1)`. In contrast, the equivalent Linux script was designed to return `exit(127)` under similar failure conditions.

2.2 Part 2: Inter-Process Communication (pingpong.c)

The `user/pingpong.c` utility spawns two distinct processes that pass a single byte back and forth leveraging two one-way pipes.

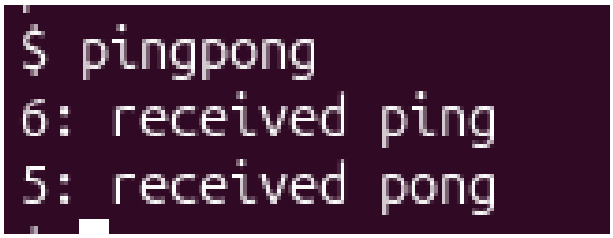


Figure 2.2: Execution of pingpong in xv6 demonstrating process synchronization

2.2.1 Observe Answers (Part 2)

1. **Why Two Pipes are Necessary:** Standard UNIX pipes are strictly unidirectional. Using two separate pipes is mandatory to avoid race conditions, ensuring that a process doesn't accidentally read the very byte it just sent before the target process gets a chance to read it.

2. **How Synchronization Works:** The parent process is kept synchronized because it encounters a blocking `read(c2p[0], &b, 1)` call. This forces the parent into a sleeping state, waking up only after the child process executes the corresponding `write(c2p[1], &b, 1)`.

3 CONCLUSIONS

Throughout this lab assignment, I confirmed that the rudimentary process management systems within the xv6 kernel behave remarkably similar to standard POSIX features found in modern Linux systems. By coding `run.c`, I gained a clearer understanding of how a shell interprets commands by sequentially chaining `fork()`, `exec()`, and `wait()`.

Furthermore, the implementation of `pingpong.c` offered hands-on experience in managing kernel-allocated pipe buffers, properly releasing file descriptors, and utilizing blocking I/O calls to perfectly synchronize process execution.